

◇ DATOS ◇ CÓDIGO ◇ MODELOS ◇ INFRAESTRUCTURA

Mejores Prácticas de MLOps

Llevar a producción sistemas de machine learning que sigan siendo confiables, reproducibles y mantenibles, mucho después de la demo.

Adaptado de un artículo de Yadidiah Kanaparthi

Usa ← → o Espacio para navegar_

CUESTIONARIO · MLFLOW

PREGUNTA 1 DE 4

¿Cuál es la diferencia principal entre lo que hace Scikit-learn y lo que hace una herramienta de seguimiento de experimentos como MLflow?

A Scikit-learn realiza el entrenamiento/predicción real; MLflow registra lo que ocurrió durante ese proceso ✓

B Scikit-learn registra metadatos; MLflow entrena el modelo

C Hacen lo mismo, solo con sintaxis diferente

D MLflow elimina por completo la necesidad de una librería de machine learning

Scikit-learn hace el trabajo real de modelado; MLflow solo registra lo que ocurrió para que puedas encontrarlo después.

CUESTIONARIO · MLFLOW

PREGUNTA 2 DE 4

En MLflow, ¿cuál de estos registrarías como un Parámetro en lugar de una Métrica?

- A La tasa de aprendizaje usada para el entrenamiento ✓
- B La precisión del modelo en el conjunto de prueba
- C Una imagen de la matriz de confusión
- D El archivo del modelo entrenado

Los parámetros son entradas que fijas antes de empezar el entrenamiento; la tasa de aprendizaje es exactamente eso.

Al tratar valores atípicos con el método del RIC (rango intercuartílico), «acotar» (winsorizar) un valor significa:

- A

Eliminar la fila que lo contiene
- B

Reemplazarlo por el límite más cercano del rango aceptable, en lugar de eliminar la fila

✓
- C

Reemplazarlo por la media de la columna
- D

Ignorarlo durante el entrenamiento pero conservarlo para la evaluación

Acotar recorta el valor hasta el límite aceptable más cercano en lugar de descartar la fila.

CUESTIONARIO · PREPROCESAMIENTO Y MÉTODOS DE ENSAMBLE

PREGUNTA 4 DE 4

Conceptualmente, ¿en qué se diferencia el Boosting (p. ej. XGBoost) del Bagging (p. ej. Random Forest)?

- A El Boosting entrena árboles de forma independiente y en paralelo; el Bagging los entrena secuencialmente
- B El Boosting entrena árboles secuencialmente, cada uno corrigiendo los errores de los anteriores; el Bagging entrena árboles de forma independiente sobre muestras bootstrap ✓
- C El Boosting solo admite tareas de regresión
- D No hay una diferencia real entre los dos enfoques

El Boosting corrige errores secuencialmente, árbol tras árbol; el Bagging promedia árboles independientes entrenados en paralelo.

¿Qué es MLOps?

MLOps aplica la disciplina de DevOps —automatización, pruebas, versionado, monitoreo— a los desafíos particulares del machine learning: datos que cambian, modelos que se degradan y experimentos que deben ser reproducibles.

FIG. 01

DevOps

Integración continua, entrega continua, infraestructura como código y pruebas automatizadas.

FIG. 02

Ingeniería de Datos

Pipelines de datos, controles de calidad, versionado y gobernanza para los datos que alimentan cada modelo.

FIG. 03

Machine Learning

Experimentación, entrenamiento, evaluación y la naturaleza estadística del comportamiento del modelo.

MLOps vs. DevOps: Qué Cambia

MLOps surgió de DevOps, pero el machine learning introduce modos de fallo que la ingeniería de software tradicional no tiene.

Principios Compartidos

- Automatización de procesos
- Integración y despliegue continuos
- Colaboración y comunicación
- Escalabilidad y confiabilidad
- Monitoreo y ciclos de retroalimentación

Exclusivo de MLOps

Reproducibilidad

El mismo código y los mismos datos aún pueden dar resultados distintos: versiona datos, semillas, hiperparámetros y entorno.

Escalar Entrenamiento e Inferencia

El entrenamiento necesita hardware especializado (GPUs); el servicio debe ser rápido y barato para que valga la pena.

Drift del Modelo

La precisión se degrada cuando los datos reales se alejan de aquello con lo que se entrenó el modelo.

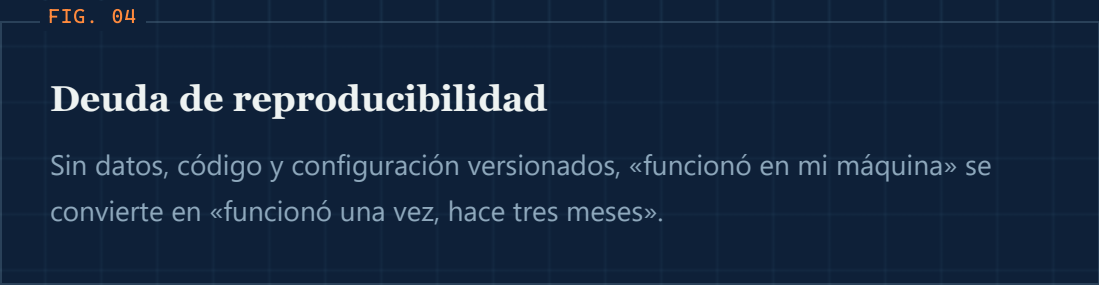
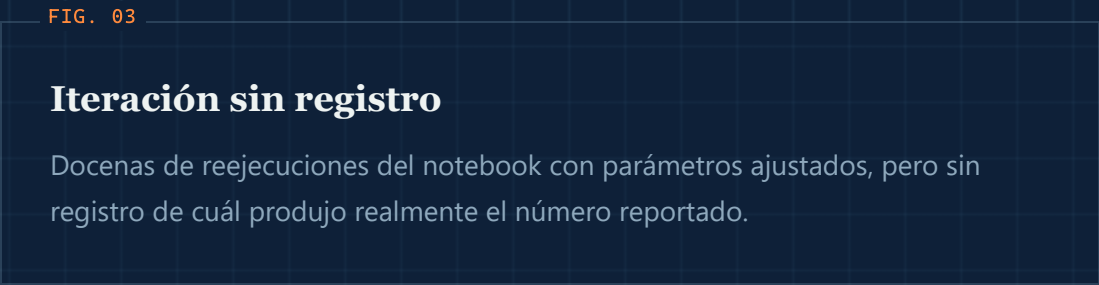
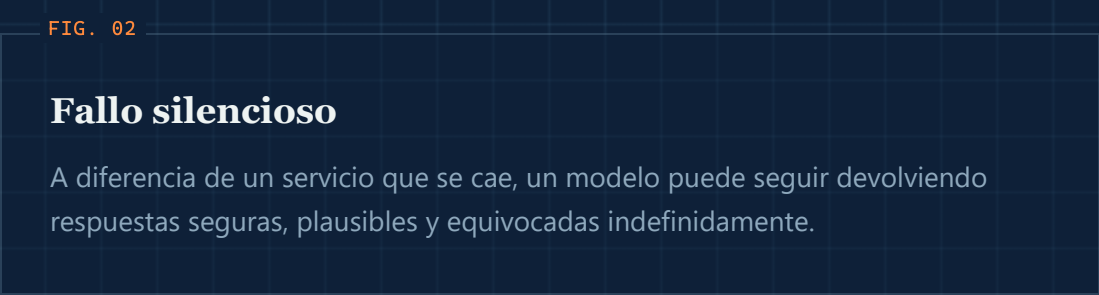
Composición del Equipo

Investigadores e ingenieros de producción tienen habilidades y ritmos distintos; deben aprender a trabajar juntos.

01 · FUNDAMENTOS

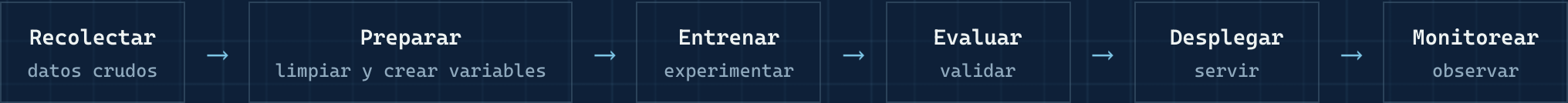
Por Qué Importa

Un notebook que produce una buena métrica no es un producto. La mayoría de las iniciativas de ML se estancan entre el prototipo y la producción, no porque el modelo esté mal, sino porque nada a su alrededor está operacionalizado.



El Ciclo de Vida del ML Es un Bucle

A diferencia del software tradicional, el ciclo de vida nunca termina realmente en el «despliegue». La retroalimentación de producción reconfigura continuamente la siguiente iteración.



↪ El reentrenamiento retroalimenta a Recolectar y Preparar

El Panorama de Herramientas de MLOps

Más allá de MLflow y DVC: un ecosistema más amplio cubre cada etapa del ciclo de vida del ML.

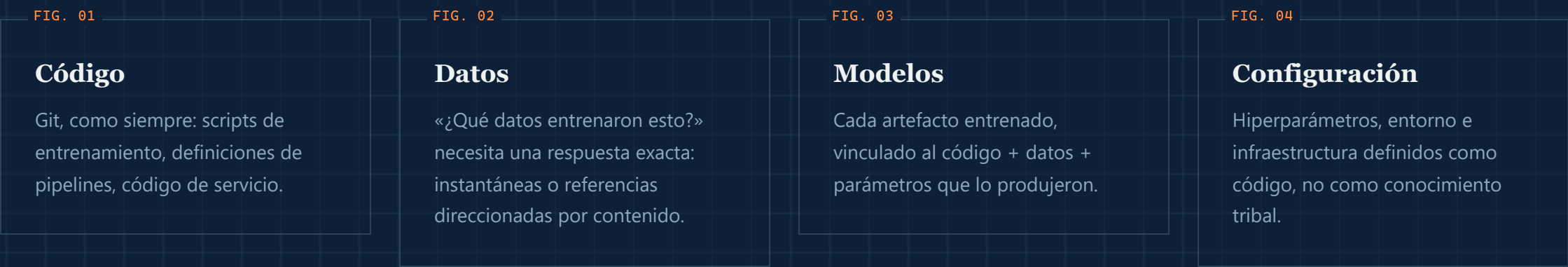


Fuente: InfluxData, «MLOps: A Comprehensive Guide to Machine Learning Operations»

02 · REPRODUCIBILIDAD

Versiona Todo

Si no puedes reconstruir exactamente qué produjo un modelo, no puedes depurarlo, auditarlo ni confiar en él. Cuatro cosas necesitan un historial de versiones.



Versionado de Datos y Linaje

La revisión de código responde «qué cambió y por qué». Los datos necesitan la misma respuesta: qué instantánea, transformada cómo, por qué proceso.

FIG. 01

Versiona como el código

Herramientas como DVC, LakeFS o Delta Lake permiten etiquetar, comparar y revertir conjuntos de datos igual que Git maneja los archivos fuente.

FIG. 02

Rastrea el linaje

Registra qué fuentes crudas, transformaciones y procesos produjeron cada tabla o variable derivada.

FIG. 03

Depura hacia atrás

Cuando un modelo se comporta mal, el linaje permite rastrear hacia atrás hasta el cambio exacto que lo causó.

FIG. 04

Habilita auditorías

Las industrias reguladas necesitan probar exactamente qué datos entrenaron un modelo en producción, meses después.

02 · REPRODUCIBILIDAD

DVC en la Práctica

Versiona conjuntos de datos y pipelines junto con Git: reproducibles por diseño.

TERMINAL

```
# versiona un conjunto de datos
dvc add data/train.csv
git add data/train.csv.dvc
git commit -m "Add training data v1"

# ejecuta el pipeline completo
dvc repro
```

DVC.YAML – DEFINICIÓN DEL PIPELINE

```
stages:
  preprocess:
    cmd: python src/data/preprocessing.py
    deps:
      - src/data/preprocessing.py
      - data/raw/transactions.csv
    outs:
      - data/processed/clean.csv
  train:
    cmd: python src/models/train.py
    deps:
      - src/models/train.py
      - data/processed/clean.csv
    metrics:
      - metrics.json
```

Cada versión del conjunto de datos está ligada a un commit de Git: al hacer checkout de cualquier commit, `dvc checkout` restaura exactamente los datos que lo produjeron.

Registra Cada Experimento

Si un resultado no se registra, no existió. El seguimiento de experimentos convierte ejecuciones improvisadas de notebooks en un historial buscable y comparable.

FIG. 01

Parámetros

Hiperparámetros, conjuntos de variables, semillas aleatorias: la receta de cada ejecución.

FIG. 02

Métricas

Curvas de pérdida, precisión, métricas de negocio: registradas paso a paso, no solo el número final.

FIG. 03

Artefactos

Pesos del modelo, gráficos e informes de evaluación adjuntos a la ejecución que los produjo.

Herramientas como MLflow o Weights & Biases existen principalmente para resolver la pregunta «¿cuál fue esa ejecución?».

03 · EXPERIMENTACIÓN

MLflow en Acción: Registra Todo

Una función de entrenamiento basada en configuración registra parámetros, métricas y el propio modelo en cada ejecución.

```
def train_model(config_path):  
    config = yaml.safe_load(open(config_path))  
    mlflow.set_experiment(config['experiment_name'])  
  
    with mlflow.start_run(run_name=config['run_name']):  
        mlflow.log_params(config['model_params'])  
  
        X_train, y_train = load_data(config['data_path'])  
        model = RandomForestClassifier(**config['model_params'])  
        model.fit(X_train, y_train)  
  
        mlflow.log_metric("accuracy",  
                          accuracy_score(y_val, model.predict(X_val)))  
        mlflow.sklearn.log_model(model, "model")
```

Compara experimentos visualmente en la interfaz de MLflow

Reproduce cualquier ejecución a partir de sus parámetros registrados y la versión del código

Despliega modelos directamente desde el registro

Entornos Reproducibles

«Funciona en mi máquina» no es aceptable para un trabajo de entrenamiento que debe ejecutarse de forma idéntica hoy, el próximo mes y en la laptop de un compañero.

FIG. 01

Contenedores

Las imágenes de Docker fijan el sistema operativo, los drivers y las librerías del sistema alrededor del código de entrenamiento/servicio.

FIG. 02

Dependencias fijadas

Versiones exactas y con hash de cada paquete, no rangos abiertos, para que una reconstrucción resuelva siempre igual.

FIG. 03

Infraestructura como código

Clústeres, GPUs y redes definidos en plantillas versionadas, no configurados a mano con clics.

03 · EXPERIMENTACIÓN

Cómo formatear su proyecto listo para Producción

Deja de usar notebooks planos, Un proyecto listo para producción separa responsabilidades: datos, código fuente, configuración y pruebas, cada uno con su propio lugar.

data/

conjuntos de datos crudos, procesados y con variables ya construidas

src/

código de datos, variables, modelos y evaluación

configs/

hiperparámetros y configuración del pipeline como código

tests/

validación automatizada de código y datos

notebooks/

solo para exploración, nunca lógica de producción

dvc.yaml

definición reproducible del pipeline

```
ml-project/
├── data/
│   ├── raw/
│   ├── processed/
│   └── features/
├── notebooks/
├── src/
│   ├── data/
│   ├── features/
│   ├── models/
│   └── evaluation/
├── configs/
├── tests/
├── mlruns/
├── dvc.yaml
└── requirements.txt
```

Probar un Sistema de ML

Las pruebas unitarias solas no detectan un modelo que está estadísticamente equivocado. Los sistemas de ML necesitan su propia pirámide de pruebas.

FIG. 01

Pruebas de datos

Verificación de esquema, rangos, nulos y distribución en cada lote entrante.

FIG. 02

Pruebas unitarias

Transformaciones de variables y código del pipeline, probados como cualquier otro software.

FIG. 03

Pruebas de comportamiento

Verificaciones de invariancia y expectativas direccionales: p. ej., cambiar un campo no relacionado no debería alterar la predicción.

FIG. 04

Pruebas de integración

El pipeline completo de extremo a extremo, incluyendo latencia de servicio y verificación de contratos.

Buenas Prácticas

Cinco hábitos que distinguen a los equipos que llevan ML a producción de forma confiable.

1

Trata la Configuración como Código

Nunca fijas los hiperparámetros en el código; usa archivos de configuración YAML versionados.

2

Automatiza los Controles de Calidad de Datos

Valida cada lote con herramientas como Great Expectations antes de entrenar.

3

Implementa Monitoreo de Modelos

Detecta el drift de datos automáticamente y dispara el reentrenamiento (p. ej. con Evidently).

4

Versiona Todo

Código en Git, datos en DVC, modelos en el Registry de MLflow, infraestructura en Terraform.

5

Despliegues en Modo Sombra

Ejecuta los modelos candidatos junto a producción y compáralos antes de promoverlos.

Errores Comunes y Cómo Evitarlos

Cuatro errores que hunden en silencio los proyectos de ML, y cómo corregir cada uno.

FIG. 01

Training/Serving Skew

PROBLEMA: Las variables se calculan de forma distinta en entrenamiento y en producción.

SOLUCIÓN: Usa un feature store o código de ingeniería de variables compartido para ambos.

FIG. 03

Ignorar la Degradación del Modelo

PROBLEMA: La precisión se degrada silenciosamente a medida que el mundo cambia.

SOLUCIÓN: Programa reentrenamientos regulares y monitorea el desempeño de forma continua.

FIG. 02

Fuga de Datos en la Validación Cruzada

PROBLEMA: Mezclar los datos antes de dividirlos deja que información futura se filtre al entrenamiento.

SOLUCIÓN: Usa divisiones basadas en el tiempo para datos temporales.

FIG. 04

Sobre-ingeniería Prematura

PROBLEMA: Construir clústeres de Kubernetes antes de validar el valor del modelo.

SOLUCIÓN: Sigue el principio gatear → caminar → correr: empieza simple y añade complejidad cuando el valor esté probado.

05 · ERRORES COMUNES

Cómo Corregir un Error Común: Divisiones Basadas en el Tiempo

Mezclar aleatoriamente es la causa más común de fuga de datos en problemas de series temporales.

INCORRECTO

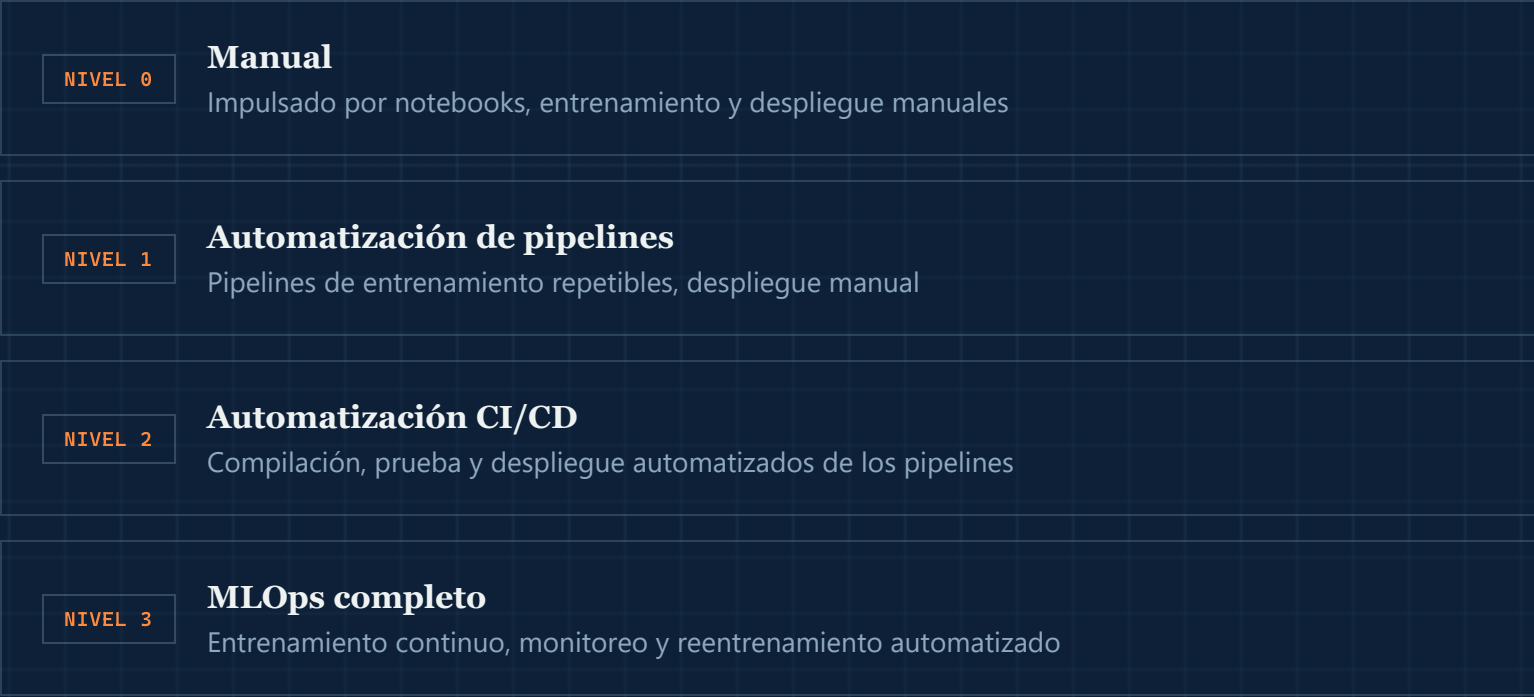
```
# Incorrecto: los datos futuros se filtran al entrenamiento
X_train, X_test = train_test_split(
    X, test_size=0.2, shuffle=True)
```

CORRECTO

```
# Correcto: dividir estrictamente por tiempo
split_date = df['date'].quantile(0.8)
train = df[df['date'] < split_date]
test = df[df['date'] ≥ split_date]
```

Un Modelo de Madurez de MLOps

La madurez es un gradiente, no un interruptor. La mayoría de las organizaciones avanza por estas etapas una capacidad a la vez.



Implementar MLOps: Una Hoja de Ruta

Un camino general que siguen las organizaciones para establecer una práctica de MLOps.

1	Establece tu punto de partida y tus objetivos Mide tu tiempo de despliegue y precisión actuales; define metas de mejora concretas.
2	Forma el equipo de MLOps Combina ciencia de datos, ingeniería, operaciones y conocimiento del negocio.
3	Define la gobernanza de datos Establece estándares de recolección, almacenamiento, versionado, calidad y cumplimiento.
4	Elige herramientas y plataformas Evalúa construir vs. comprar, la experiencia del equipo y el soporte de la comunidad.
5	Automatiza el pipeline de despliegue Empieza con CI/CD para probar, rastrear y promover modelos.
6	Itera y mejora MLOps es continuo: revisa los objetivos y sube el estándar con el tiempo.

CIERRE

Conclusiones Clave

FIG. 01

Versiona todo

Código, datos, modelos y configuración, todo reconstruible.

FIG. 02

Automatiza el pipeline

Desde la validación de datos hasta el despliegue, no solo el entrenamiento.

FIG. 03

Monitorea en producción

El drift y la degradación son la norma; vigílalos de forma continua.

FIG. 04

Comparte la responsabilidad

MLOps funciona como una práctica de equipo, no como una entrega.

Gracias.